

* Types of argument / Parameters Passing technique :

1) Positional arguments $\left\{ \begin{array}{l} \text{non default (sequenced Positional arg)} \\ \text{default} \end{array} \right.$

2) Keyword arguments (key word - only)

3) Arbitrary arguments $\left\{ \begin{array}{l} \text{Arbitrary Positional/variable args (*arg)} \\ \text{Arbitrary Keyword args (**kwarg)} \end{array} \right.$

means $\Rightarrow$ (multiple) (* symbol)

(*arg) (variable Positional)

n no. of args (values)

(keyword) (format : Para = arg)

(**kwargs)

n no. of kwargs

1) Positional Argument :

It is a technique in which no. of Parameters defined during function definition must be equal to no. of arguments Passed during function call.

a) non-default argument : (required Positional argument)

- > These arguments must be Passed during the function call
- > If we miss them to Pass, Python throws error

> eg:- `def add_nums(a,b) :`
`Print(a+b)`

<code>add_nums()</code>	<code># Type Error</code>
<code>add_nums(2)</code>	<code># Type Error</code>
<code>add_nums(2,3,4)</code>	<code># Type Error</code>
<code>add_nums(2,2)</code>	<code># 6</code>

b) Default arguments :

- > During the function definition if a Parameter is defined with a value to it, then those type of Parameters are called as default Parameters and those values (arguments) are called as default arguments.

> eg:- `def add_nums(a,b=5) :`
`Print(a+b)`

<code>add_nums(2)</code>	<code># 7</code>
<code>add_nums(3,7)</code>	<code># 10</code>

• Note :-

- > In the 1st funⁿ call only the value for 'a' is Passed hence 'b' will take the value as '5' (default value) and give the sum as '7'
- > In the 2nd funⁿ call 'a' will takes the value as '3' and even though 'b' is having a default value it will be overridden by a value which is Passed during funⁿ call
- > eg 2 :- `def add_nums (a=4, b) :`
`Print (a+b)`

`add_nums (2)`

Syntax Error : Parameter without a default follow
Parameter with a default.

• Note :-

- > In the above function it returns Syntax Error because a non-default Parameter can't follow a default Parameter
- > In Positional argument technique, order matters
- 2) Keyword Arguments : (Keyword only)
- > During the function call if a argument is Passed along with it's Parameters name then it is called as Keyword argument.

- note :- Key = word
variable = value
Parameter = arguments

no order matters

- Format :- Parameter = argument

- eg 1 :-

```
def info(name, age, gender, loc) :  
    Print(f"Name : {name}")  
    Print(f"Age : {age}")  
    Print(f"Gender : {gender}")  
    Print(f"loc : {loc}")
```

 } -> optional

```
info(name="Masha", age=12, gender="F", loc="goa")  
info(age=12, name="Masha", gender="F", loc="mysore")
```

- note :- In this technique order does not matters

- eg 2 :-

```
def info(name, age, gender, loc="Goa") :  
    Print(name, age, gender, loc)
```

```
info(name="masha", age=23, gender="F")
```

```
# Masha  
23  
F  
Goa
```

```
info("John", age=23, gender="M")
```

```
# John  
23  
M  
Goa
```

info(name="John", 34, gender="M") # Syntax Error

note:- In the above function call it returns Syntax Error because Positional arguments cannot follow the Keyword argument

note:- non-default ^{mandatory} default → Key

3) Arbitrary arguments :-

These are the arguments which allows a function to accept n no. of arguments

1) Arbitrary variable / Positional Arguments :- (*args)

> These are the arguments, which allows a function to accept any no. of Positional values

> To Provide multiple arguments, there must be a Parameter defined with a '*' Prefixed to it ^{during} ~~defining~~ function definition

> Syntax:-

```
def fun_name(*Para):  
    # statements  
fun_name(n no. of values)
```

> Eg:-
def fun(*a):
 Print(a, type(a))

fun()

fun(12)

fun(12, 90, "Py", 54)

() <class 'tuple'>

(12,) <class 'tuple'>

(12, 90, 'Py', 54) <class 'tuple'>

> note:-

- The values are stored in the form of tuple
- There can be only one *arg present in a function definition

ii) Arbitrary Kwargs :- (** Kwargs) :

- > These are the args which allows a function to accept any no// of Kwargs.
- > To provide multiple Kwargs, there must be a parameter defined with ** Prefix to it during the function definition

> Syntax:-

```
def fun_name (**Para):
    # statements
    fun_name (n no// of Kwargs)
```

> eg:-

```
def fun (**a):
    print(a, type(a))

fun() # {}
fun(o="one", b1="bat") # {'o': 'one', 'b1': 'bat'}
```

- > All the Key value Pairs are stored in the form of a dictionary.
- > There can be only one ** Kwargs present in a function definition.

Imp ★ order of arguments:-

- non - default
- default
- * args
- Keyword - only arg
- ** Kwargs

eg:- def std-info(name, gender, *Phone-no, loc, **Marks) :

Print(name)

Print(gender)

Print(Phone-no)

Print(loc)

Print(Marks)

std-info("John", "M", 9876543210, 9960220481, loc="Bangalore",
Kam=45, Eng=49, Hin=45)

> eg:- def std-info(name, gender, age=23, *Phone-no, loc, **marks) :

Print(f" name : {name}")

Print(f" gender : {gender}")

Print(f" age : {age}")

Print(f" Phone-no : {Phone-no}")

Print(f" location : {loc}")

Print(f" marks : {marks}")

std-info("John", "M", 24, 9876543210, 9876542312, loc="Bangalore",
Kam=50, Eng=65, Hin=48)